SonicGen v2.0

Advanced Multimodal Gaming Audio AI

Final Challenge | Prathyu Adari | 16371669

COMP_SCI-5588-0001-14650-2026SP-DS Capstone

Problem Statement

Static Audio Experiences:

Modern games use repetitive, pre-recorded soundtracks that fail to adapt to real-time player emotions.

One-Dimensional Profiling:

Player analysis currently relies on telemetry (K/D ratios) rather than behavioral or communication nuance.

Market Need:

Players demand deeper immersion, while streamers need copyright-free, reactive music for live audiences.

The SonicGen Solution:

A system that 'listens' to the player and adapts the sonic environment instantly.

Week 14 Summary: The Foundation

Conceptual Proof:

Chained Whisper-small and MusicGen-small in a basic script.

Basic Input:

Transcribed voice chat and generated a single music prompt.

Key Findings:

Demonstrated that semantic keywords (e.g., 'attack') could trigger specific musical moods.

Limitations:

High latency, no safety features, and a text-only output terminal.

Final Enhancements: SonicGen v2.0

Interactive Web App:

Implemented a Gradio-based GUI for easy recording and visualization.

Deep Multimodal Pipeline:

Added Text-to-Speech (TTS) for an AI Companion voice response.

Optimization:

Transitioned to FP16 precision and GPU acceleration for faster inference.

Quantitative Evaluation:

Integrated WER (Word Error Rate) tracking and latency monitoring.

Safety:

Added a real-time toxicity filter for responsible AI deployment.

Advanced Pipeline Architecture

Step 1:

Voice Capture -> Standardized audio buffer via Gradio.

Step 2:

Whisper-Small (STT) -> Transcription of gaming intent.

Step 3:

Behavioral Logic & Toxicity Check -> DNA classification + safety gate.

Step 4 (Parallel Output A):

MusicGen -> 5-second contextual battle/stealth loop.

Step 5 (Parallel Output B):

gTTS -> AI companion vocal feedback.

Step 6:

Evaluation Layer -> WER calculation and latency logging.

Foundation Models & Technology

openai/whisper-small:

Selected for its balance between transcription accuracy and inference speed.

facebook/musicgen-small:

Optimized for generating high-quality musical segments based on semantic prompts.

gTTS (Google Text-to-Speech):

Utilized for creating clear, low-latency agent voice responses.

Jiwer:

Library used for calculating Word Error Rate (WER) against true transcripts.

Engineering & Performance

GPU Acceleration:

Models deployed on CUDA-enabled T4 GPUs to minimize processing bottlenecks.

Half-Precision (FP16):

Implemented to reduce memory footprint and increase inference speed by ~40%.

Batch Logic:

Simplified behavioral extraction using keyword-mapping to minimize LLM-token overhead.

Asynchronous Generation:

Preparation of TTS and Music assets in a multi-threaded capable structure.

Productization: Gradio Interface

Clean UX:

Simple microphone input and visual waveform feedback.

Real-time Results:

Dashboard showing transcription, extracted DNA, and engineered prompt.

Dual Audio Playback:

Integrated players for the companion voice and the custom soundtrack.

Analytics Panel:

Dedicated section for latency and accuracy (WER) reporting.

Prompt Engineering Strategy

Behavioral Mapping:

8-Axis DNA traits mapped to music theory parameters (BPM, Scale, Instrument).

Aggressive DNA -> 'High energy, heavy bass, fast tempo 140bpm, aggressive synth, cyberpunk'.

Stealth DNA -> 'Dark ambient, tense strings, slow tempo, quiet atmospheric'.

Input Filtering:

Pre-processing to remove filler words and focus on tactical keywords.

Evaluation & Metrics

Word Error Rate (WER):

Quantitative measure of Whisper's transcription success.

Latency Benchmarking:

Measuring end-to-end time from speech-end to audio-start.

Alignment Score:

Qualitative human rating of music mood vs. voice intent.

Diversity Check:

Ensuring the model doesn't repeat identical loops for different players.

Results: Speech Performance

Average WER:

~12% on standard commands; increased to 18% with heavy background noise.

Slang Challenges:

Models initially struggled with terms like 'Gank' or 'DPS' without fine-tuning.

Robustness:

Excellent performance on accent diversity and sentence structure.

Accuracy:

Higher than 90% for core tactical identification (Push/Defend/Stealth).

Results: Audio & Speed

Prompt Alignment:

92% of users reported music matched their intent in blind testing.

Latency:

Average end-to-end time of 12.4 seconds on T4 GPU.

Optimization Insight:

Limiting MusicGen output to 5 seconds significantly improved user experience over 10-second generations.

Comparison: MVP vs. Final v2.0

Week 14 Baseline:

Text terminal output only | No safety/toxicity checks | ~30s latency

Final SonicGen v2.0:

Full Web GUI | Integrated Safety Guardrails | ~12s optimized latency

Multimodal Companion (TTS) | Quantified Evaluation metrics (WER)

Business & Monetization

Target Users:

Indie game studios, Twitch/YouTube streamers, eSports platforms.

Market Need:

$200B+ gaming market seeking 'Dynamic Personalized Experiences'.

Pricing Model:

SaaS API (Credits per generation minute) or Plugin Licensing (per-seat).

Unique Value:

Copyright-free, infinite variation music that is reactive to playstyle.

Responsible AI: Toxicity Filter

Logic:

Real-time screening of transcription against a harmful language database.

Action:

If toxicity is detected, the pipeline terminates and the AI Companion issues a warning.

Purpose:

Prevents the AI from rewarding abusive behavior with 'aggressive' power-trip soundtracks.

Compliance:

Adheres to standard 'Safety & Misuse Prevention' guidelines.

Risks & Ethics

Dataset Bias:

MusicGen may favor specific western musical tropes; mitigation via diverse style prompts.

Voice Cloning:

Avoided deepfake risks by using generic, synthetic TTS voices.

Data Privacy:

Voice chat is processed in memory; no persistent storage of private conversations.

Copyright:

Reliance on open-source weights (MusicGen) ensures non-infringing outputs.

Limitations & Future Work

Current Limit:

Latency is still too high for instantaneous twitch-reflex FPS gameplay.

Future Goal A:

Integrate AudioLDM2 for specific sound effects (custom footsteps/gunshots).

Future Goal B:

Model Distillation to allow local, zero-latency execution on player hardware.

Future Goal C:

Unreal Engine / Unity native plugins for commercial game deployment.

Conclusion

SonicGen v2.0 proves that multimodal AI can create a truly adaptive 'living' game environment.

By bridging Speech, Text, and Sound, we provide a glimpse into the future of interactive entertainment.

Thank you for your time. Links to the GitHub Repository and Video Demo are available in the submission notes.